

Méthodologie de commande avancée : Commande prédictive

Florent Bergé, Léa Grousseau

28 mars 2026

1 Introduction

L'objectif de ce TP est de se familiariser la commande prédictive (ou Model Predictive Control, MPC) en l'appliquant sur l'étude de cas du modèle des quadruples réservoirs (QTP). Ce système est composé de 4 réservoirs reliés entre eux deux à deux tels que représentés ci-dessous (cf fig 1 et alimentés par 2 pompes).

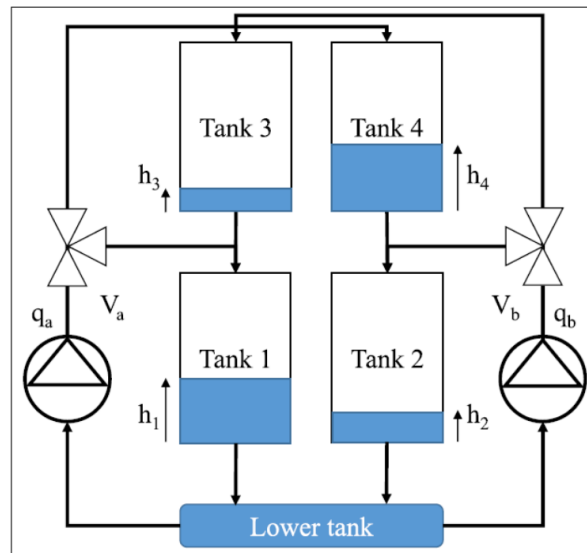


FIGURE 1 – Système des 4 réservoirs

On cherche à asservir la hauteur d'eau des deux réservoirs du bas grâce à la commande des 2 pompes en utilisant notamment la toolbox CasADI de Matlab.

2 Processus et modélisation

2.1 Modélisation

2.1.1. Sur ce système, on contrôle le débit des deux pompes A et B qui sont donc nos entrées de commande : $u = \begin{bmatrix} q_a \\ q_b \end{bmatrix}$. Les sorties à réguler sont les hauteurs d'eau des réservoirs 1 et 2 notées $y = \begin{bmatrix} h_1 \\ h_2 \end{bmatrix}$. Enfin les états du système sont les hauteurs des quatre réservoirs soit $x = [h_1 \ h_2 \ h_3 \ h_4]^T$.

2.1.2. Les équations régissant le système obtenues grâce aux lois de la mécanique des fluides sont les suivantes :

$$\frac{dh_1}{dt} = -\frac{a_1\sqrt{2gh_1}}{S} + \frac{a_3\sqrt{2gh_3}}{S} + \frac{\gamma_a q_a}{S} \quad (2.1)$$

$$\frac{dh_2}{dt} = -\frac{a_2\sqrt{2gh_2}}{S} + \frac{a_4\sqrt{2gh_4}}{S} + \frac{\gamma_b q_b}{S} \quad (2.2)$$

$$\frac{dh_3}{dt} = -\frac{a_3\sqrt{2gh_3}}{S} + \frac{(1-\gamma_b)q_b}{S} \quad (2.3)$$

$$\frac{dh_4}{dt} = -\frac{a_4\sqrt{2gh_4}}{S} + \frac{(1-\gamma_a)q_a}{S} \quad (2.4)$$

On observe que le système est non linéaire car $\dot{h}_1$ dépend de $\sqrt{h_1}$ et non de h_1 (de même pour les 3 autres variables d'état). De plus le système est couplé car $\sqrt{h_3}$ et $\sqrt{h_4}$ interviennent respectivement dans les équations régissant h_1 et h_2 .

2.1.3. En régime permanent on a $\frac{d}{dt} \equiv 0$. A partir des quatre équations précédentes on en déduit les expressions de $q_{a,ref}$, $q_{b,ref}$, $h_{3,ref}$ et $h_{4,ref}$:

$$q_{a,ref} = \frac{(1-\gamma_b)a_2\sqrt{2gh_{2,ref}} - \gamma_b a_1\sqrt{2gh_{1,ref}}}{1-\gamma_a-\gamma_b} \quad (2.5)$$

$$q_{b,ref} = \frac{(1-\gamma_a)a_1\sqrt{2gh_{1,ref}} - \gamma_a a_2\sqrt{2gh_{2,ref}}}{1-\gamma_a-\gamma_b} \quad (2.6)$$

$$h_{3,ref} = \frac{\left(\frac{1-\gamma_b}{a_3}q_{b,ref}\right)^2}{2g} \quad (2.7)$$

$$h_{4,ref} = \frac{\left(\frac{1-\gamma_a}{a_4}q_{a,ref}\right)^2}{2g} \quad (2.8)$$

On nous donne les hauteurs des réservoirs 1 et 2 qui sont : $h_{1,ref} = 0.8m$ et $h_{2,ref} = 0.6m$. On peut ainsi faire l'application numérique et obtenir les valeurs suivantes :

$q_{a,ref}$	$3,44 \cdot 10^{-4} \text{ m}^3/\text{s}$
$q_{b,ref}$	$6,93 \cdot 10^{-4} \text{ m}^3/\text{s}$
$h_{3,ref}$	$1,03$
$h_{4,ref}$	$0,38$

TABLE 1 – Valeurs numériques en régime permanent

2.1.4. Nous n'avons que 2 signaux de commande. nous ne pouvons donc que réguler 2 états.

2.1.5. Un développement de Taylor au premier ordre donne :

$$\dot{x} = f(x_{ref}, u_{ref}) + \frac{\partial f}{\partial x_{x_{ref}, u_{ref}}} (x - x_{ref}) + \frac{\partial f}{\partial u_{x_{ref}, u_{ref}}} (u - u_{ref})$$

Nous avons quatre variables d'états et deux entrées de commandes donc $\frac{\partial f}{\partial x x_{ref}, u_{ref}}$ est une matrice jacobienne $J_f(x)$ de taille 4x4 et $\frac{\partial f}{\partial u x_{ref}, u_{ref}}$ est la matrice $J_f(u)$ de taille 4x2 dont les expressions sont :

$$J_f(u) = \frac{1}{S} \begin{bmatrix} \gamma_a & 0 \\ 0 & \gamma_b \\ 0 & 1 - \gamma_b \\ 1 - \gamma_a & 0 \end{bmatrix} \quad (2.9)$$

$$J_f(x) = \frac{1}{S} \sqrt{\frac{g}{2}} \begin{bmatrix} -\frac{a_1}{\sqrt{h_1}} & 0 & \frac{a_3}{\sqrt{h_3}} & 0 \\ 0 & -\frac{a_2}{\sqrt{h_2}} & 0 & \frac{a_4}{\sqrt{h_4}} \\ 0 & 0 & -\frac{a_3}{\sqrt{h_3}} & 0 \\ 0 & 0 & 0 & -\frac{a_4}{\sqrt{h_4}} \end{bmatrix} \quad (2.10)$$

2.1.6. En posant $x' = (x - x_{ref})$ et $u' = (u - u_{ref})$ on obtient l'équation :

$$\dot{x} = Ax' + Bu'$$

avec $A = J_f(x)$ et $B = J_f(u)$. On calcule le rang de la matrice de commandabilité donnée par $\begin{bmatrix} B & AB & A^2B \end{bmatrix}$.

```
% Initialisation des parametres
a1 = 1.31e-4;
a2 = 1.51e-4;
a3 = 9.27e-5;
a4 = 8.82e-5;
ga = 0.3;
gb = 0.4;
Sc = 0.06;
g = 9.81;

% Points de reference
h1ref = 0.8;
h2ref = 0.6;

qbref = ((1-ga)*a1*sqrt(2*g*h1ref)-ga*a2*sqrt(2*g*h2ref))/(1-ga-gb);
qaref = (-gb*a1*sqrt(2*g*h1ref)+(1-gb)*a2*sqrt(2*g*h2ref))/(1-ga-gb);
h3ref = ((1-gb)*qbref)^2/(2*g*a3^2);
h4ref = ((1-ga)*qaref)^2/(2*g*a4^2);

% Jacobiennes analytiques
Alin = (1/Sc)*sqrt(g/2).*[-a1/sqrt(h1ref) 0 a3/sqrt(h3ref) 0;
    0 -a2/sqrt(h2ref) 0 a4/sqrt(h4ref);
    0 0 -a3/sqrt(h3ref) 0;
    0 0 0 -a4/sqrt(h4ref)];

Blin = (1/(3600*Sc)).*[ga 0; %
    0 gb;
    0 1-gb;
    1-ga 0];
```

```
rank(ctrb(Alin, Blin)); % =4 donc commandable
```

Le rang de la matrice de commandabilité est 4, ce qui correspond à l'ordre de notre système. D'après le système de Kalman, le système est commandable.

2.1.7. On conçoit un contrôleur LQ sur le modèle linéarisé et on le compare au modèle du système réel fourni avec le schéma suivant sur Simulink (cf fig 2).

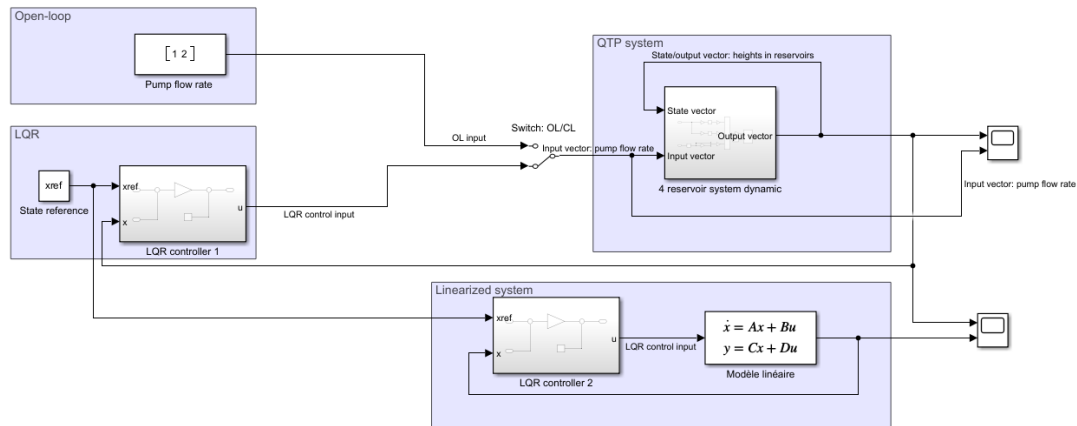


FIGURE 2 – Schéma Simulink du système réel et de notre modèle linéarisé

Le contrôleur du modèle linéarisé nous permet d'obtenir la commande u à partir de x et x_{ref} tel que suit (cf fig.3), avec la valeur de K obtenue grâce à la fonction `lqr` de Matlab.

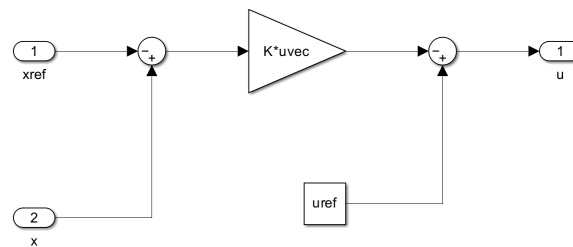


FIGURE 3 – Enter Caption

```
xmax = 1.36*ones(4,1);
umax = [3.26; 4];

Q = diag(1 ./ (xmax.^2));
R = diag(1 ./ (umax.^2));

[K,~,~] = lqr(A, B, Q, R);
```

On récupère les valeurs des variables d'états et des commandes du modèle du système réel (cf fig.5).

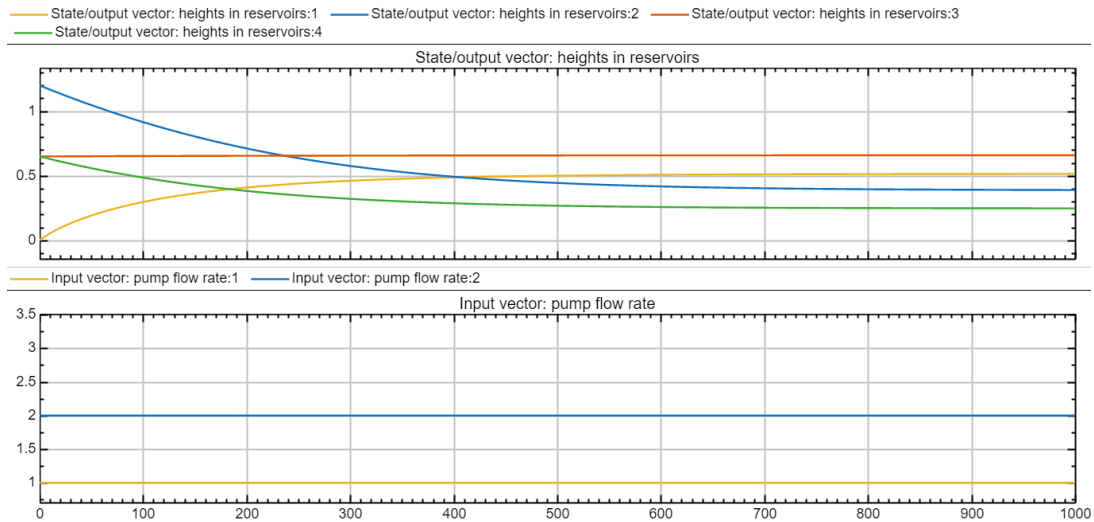


FIGURE 4 – Simulation du modèle en boucle ouverte

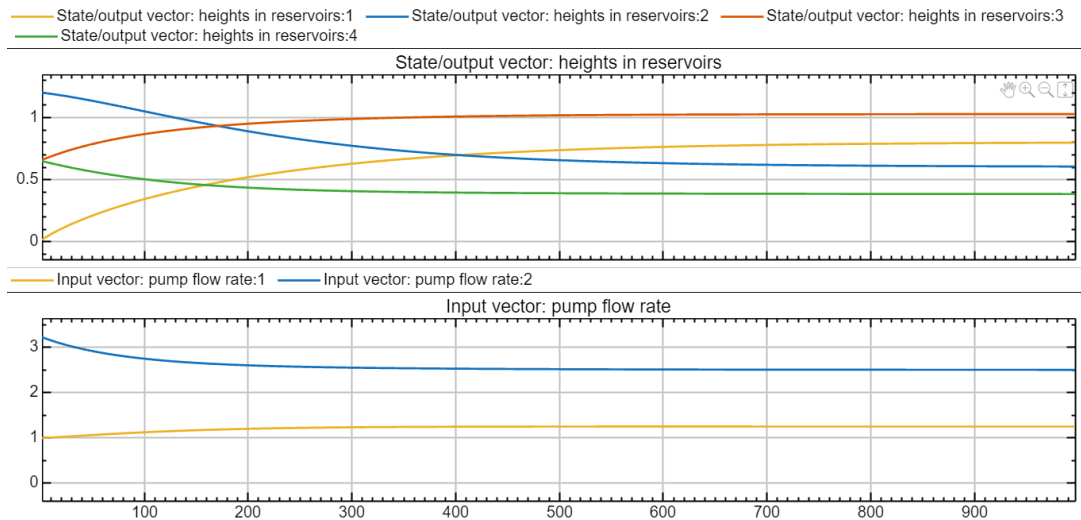


FIGURE 5 – Simulation du modèle linéarisé

On observe que les commandes et les variables d'états convergent vers les

valeurs $x_{ref} = \begin{bmatrix} 0.8 \\ 0.6 \\ 1.03 \\ 0.38 \end{bmatrix}$ et $u_{ref} = \begin{bmatrix} 3,44 \cdot 10^{-4} \text{ m}^3/\text{s} \\ 6,93 \cdot 10^{-4} \text{ m}^3/\text{s} \end{bmatrix} = \begin{bmatrix} 1.24 \text{ m}^3/\text{h} \\ 2.49 \text{ m}^3/\text{h} \end{bmatrix}$ calculées précédemment pour le régime permanent.

En comparant les variables d'états de notre modèle linéaire et du modèle du système réel fourni on remarque des écarts assez importants (cf fig.6).

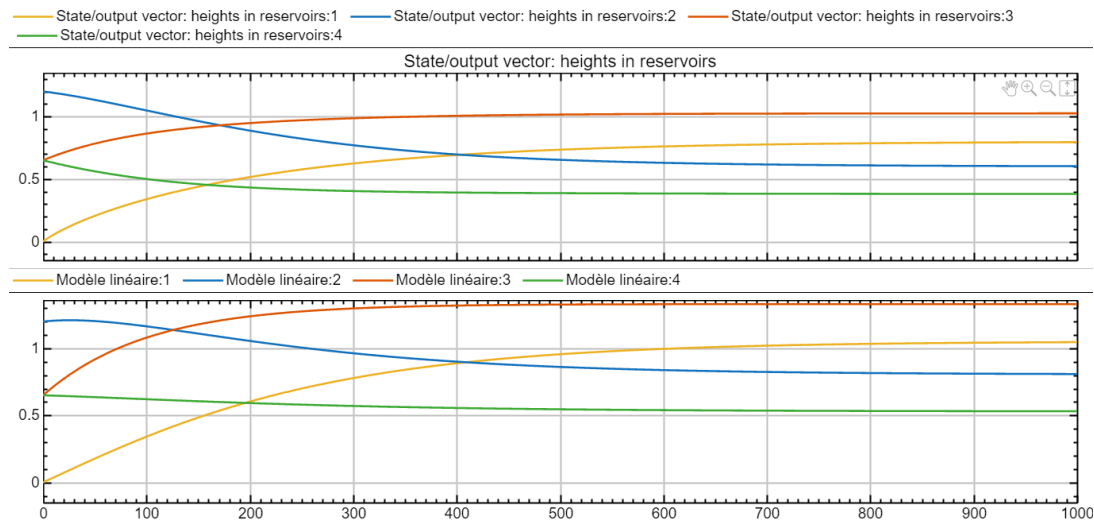


FIGURE 6 – Comparaison des vecteurs d'état obtenus avec le modèle linéarisé (en bas) avec ceux du modèle réel (en haut)

L'état initial du système se situe loin du point d'équilibre calculé autour duquel se fait le développement linéaire. Ainsi, même si les comportements asymptotiques des variables d'états suivent ceux du système réels, il y a un écart important dans les valeurs obtenues en régime permanent.

2.2 Modélisation étendue

2.2.1. Les débit q_a et q_b des pompes n'intervenant pas dans les coefficients des matrices jacobiennes, une imprécision des actionneurs ne change pas la commandabilité du système. On obtient l'équation d'état suivante, avec δu l'imprécision sur la commande :

$$\begin{aligned}\dot{x} &= A \cdot (x - x_{ref}) + B \cdot (u + \delta u - u_{ref}) \\ d &= \delta u\end{aligned}$$

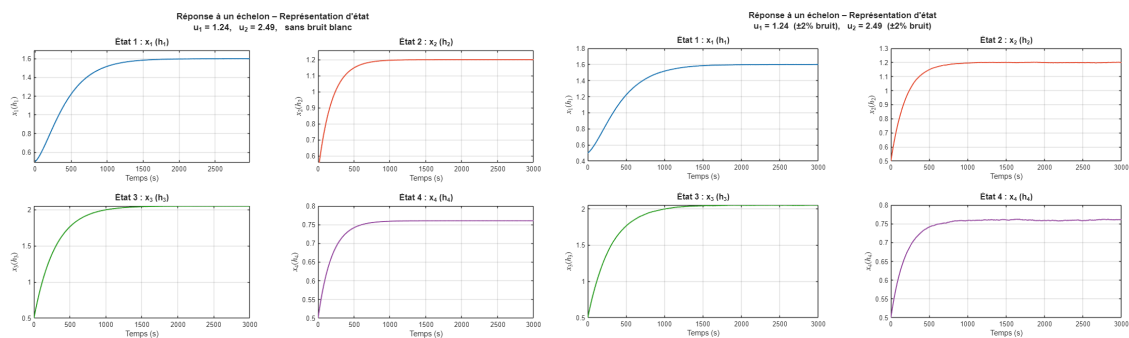


FIGURE 7 – Réponse impulsionnelle du système avec (à gauche) et sans (à droite) les imprécisions sur la commande

L'impact sur les variables d'état avec et sans l'imprécision des commandes (modélisée par un bruit blanc) est négligeable comme illustré dans la figure 7.

2.2.2. On rajoute une incertitude de 5% sur la surface de l'orifice de fuite du réservoir 1. On recalcule les matrices jacobienues et le rang de la nouvelle matrice de commandabilité du système. On obtient toujours un rang égal à 4 donc le système est toujours commandable. Les valeurs singulières et les valeurs propres obtenues avec la nouvelle valeur de a_1 sont presque identiques à la modélisation déjà réalisée. L'impact d'une surestimation est donc infime sur le niveau de h_1 en régime permanent.

2.2.3. En cas de fuite sur h_3 , il faut que le débit q_b compense cette fuite pour maintenir la stabilité de h_1 et h_2 . Pour adapter notre modèle, on pourrait par exemple modifier la valeur de γ_b

3 Commande prédictive non linéaire avec CasADI

3.1 Commande prédictive non linéaire

3.1.1. L'objectif de contrôle de QTP est de minimiser un critère sous contrainte. On propose la fonction de coût suivante :

$$J(h, q) = \int_0^{t_f} (h - h_{ref})^T Q (h - h_{ref}) + (q - q_{ref})^T R (q - q_{ref}) \quad (3.1)$$

avec $Q = \text{diag}(\frac{1}{(\max_{h_i} |h_i - h_{i,ref}|)^2})$ et $R = \text{diag}(\frac{1}{(\max_{q_i} |q_i - q_{i,ref}|)^2})$

3.1.2. La contrainte d'égalité suivante (3.2) correspond à la dynamique du modèle non linéaire du système. La contrainte de non-égalité (3.3) traduit les valeurs minimales et maximales demandées pour les hauteurs d'eau (limite d'état) et les commandes (limite d'entrée).

$$G(h, q) = h_{k+1} - f(h_k, q_k) = 0 \quad (3.2)$$

$$H(h, q) = \begin{pmatrix} h_{max} - h \\ h - h_{min} \\ q_{max} - q \\ q - q_{min} \end{pmatrix} \geq \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} \quad (3.3)$$

3.1.3. On différencie les variables qui sont renouvelées à chaque pas de temps de contrôle, celles qui restent constantes et celles qui dépendent des résultats

variables renouvelées	variables constantes	
h_1	a_1	
h_2	a_2	
h_3	a_3	
h_4	a_4	
q_a	S_c	
q_b	γ_a	
	γ_b	
		variables à optimiser
		q_a
		q_b

TABLE 2 – Classement des différentes variables lors de l'optimisation

3.1.4. On ajoute $(u - u_{ref})^T R(u - u_{ref})$ à la fonction de coût, ce qui revient à vouloir atteindre $u = u_{ref}$ en régime permanent. Cela ne correspond pas à notre objectif, qui est d'atteindre les valeurs $h_{1,ref}$ et $h_{2,ref}$ tout en minimisant l'énergie de commande (proportionnelle à u^2). On peut donc avoir u inférieur à u_{ref} . Ajouter $u^T R u$ suffit.

3.2 CasADI et mise en œuvre

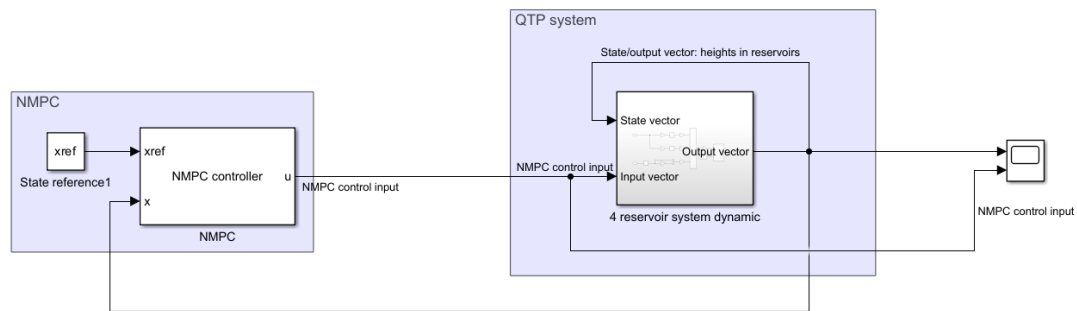


FIGURE 8 – Implémentation NMPC sur Simulink

Nous complétons le contrôleur NMPC avec le modèle du système :

Extrait de Exp2_QTP_NMPC/MPC_controller.m

```
[...]
function [opti, x, x0, xref, u] = construct_problem(obj)
    opti = casadi.Opti();
    opti.solver('ipopt',...
                struct('print_time', false), ...
                struct('print_level', 0, 'max_iter', 100));

    % Define decision variables
    % **Define state variable x here**
    x = opti.variable(obj.nx, obj.N + 1);
    % **Define input variable u here**
    u = opti.variable(obj.nu, obj.N);
```

```

x0 = opti.parameter(obj.nx, 1);
opti.subject_to( x(:, 1) == x0 );

xref = opti.parameter(obj.nx, 1);

cost = 0;
for i=1:obj.N
    ex = x(:, i) - xref;
    eu = u(:, i);

    % Stage cost (LQ-inspired)
    % J = int(x'Qx + u'Ru)
    cost = cost + ex'*obj.Q*ex + eu'*obj.R*eu;

    % Constraint
    x_next = obj.model(x(:, i), u(:, i));
    opti.subject_to( x(:, i+1) == x_next );

    % State bounds
    opti.subject_to(obj.xmin <= x(:, i));
    opti.subject_to(x(:, i) <= obj.xmax);

    % Input bounds
    opti.subject_to(obj.umin <= u(:, i));
    opti.subject_to(u(:, i) <= obj.ymax);
end

ex_N = x(:, obj.N+1) - xref;
cost = cost + ex_N' * obj.P * ex_N;

opti.minimize(cost);
end

function [uopt, time_exec, U, X] = solve(obj, x0, ref, varargin)
    tstart=tic;

    obj.opti.set_initial(obj.opti_x, repmat(x0,1,obj.N+1));
    obj.opti.set_initial(obj.opti_u, zeros(obj.nu,obj.N));
    % Set optimization parameters
    % **Set feedback as initial state**
    obj.opti.set_value(obj.opti_x0, x0(:)); % current state
    obj.opti.set_value(obj.opti_ref, ref(:)); % reference

    % **Set state reference**
    if obj.use_warm_start && ~isempty(obj.prev_sol)
        try
            obj.opti.set_initial(obj.prev_sol.value_variables());
        catch
    
```

```

        % default
    end
end

% Solve and retrieve values
sol = obj.opti.solve();
obj.prev_sol = sol;

% Extract solution
X = sol.value(obj.opti_x); % [nx x (N+1)] predicted states
U = sol.value(obj.opti_u); % [nu x N ] optimal inputs

uopt = U(:, 1);           % [nu x 1]

time_exec = toc(tstart);
end

```

3.2.1. On exécute l'implémentation NMPC (cf fig. 8) et on obtient les résultats suivants :

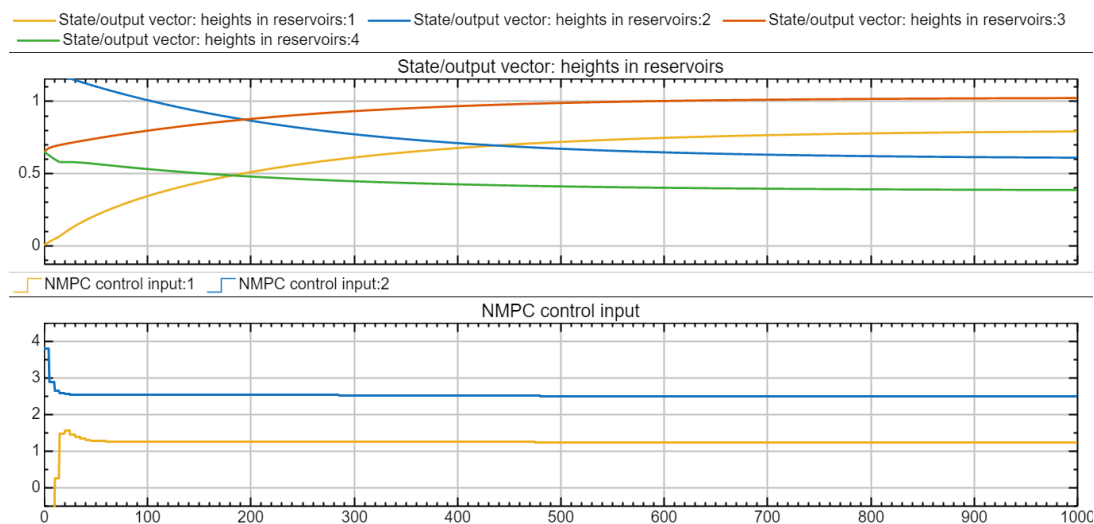


FIGURE 9 – Simulation NMPC

Les variables d'état et les commandes convergent bien toutes vers les valeurs de référence calculées précédemment. Le temps de réponse pour les variables d'état est d'une dizaine de minutes, contre quelques dizaines de secondes seulement pour les variables de commande. Les hauteurs d'eau et les débits des pompes restent entre les valeurs maximales et minimales fixées.

N.B. : Le simulateur ne trouve pas de solution et déclare une erreur pour des pondérations similaires sur Q et R . Nous modifions le code comme suit :

```

MPC.R    = eye(2);      % Weighting on state vector
MPC.Q    = 1e4*eye(4); % Weighting on input vector

```

3.2.2. La valeur $q_a(0^+)$ est négative, ce qui est impossible d'après nos contraintes fortes. De plus, les états tendent pas vers leur valeur de référence sans pondération entre Q et N comme expliqué en 3.2.1.

3.2.3. Modifions la contrainte sur les états. Pour ce faire, nous n'allons pondérer que h_1 et h_2 (autrement dit sur y_{ref}) en modifiant Q . Nous remplaçons $Q = I_4$ par :

$$Q = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \text{ (figure 10) ; puis } Q = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \text{ (figure 11)}$$

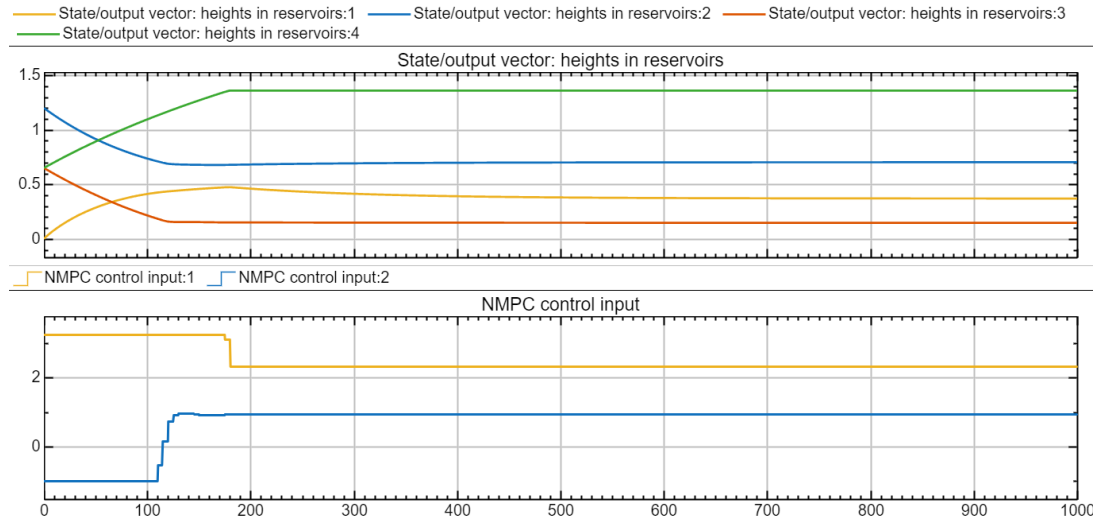


FIGURE 10 – Trajectoire pour une pondération similaire entre h_1 et h_2



FIGURE 11 – Trajectoire pour un rapport 10 de pondération entre h_1 et h_2

On observe que le réservoir 4 sature et que les valeurs finales ne correspondent pas à celles attendues en régime permanent.

Dans le premier cas, seule h_2 tend vers $h_{2,ref}$.

3.2.4. Le réservoir déborde, donc le correcteur ne remplit pas le cahier des charges. Même si le coût est élevé, il reste plus intéressant de suivre x_{ref} plu-

tôt que y_{ref} .

3.2.5. A l'aide du Simulink profiler, nous obtenons les mesures suivantes.

Modèle	Temps total d'exécution	Temps d'exécution du bloc régulateur
Boucle ouverte	22.0 s	
LQR	26.1 s	1.23 s
NMPC	31.2 s	6.76 s

TABLE 3 – Comparaison des temps d'exécution

Ces résultats ne sont pas surprenants, le nmpc doit déterminer une trajectoire à partir du modèle non linéaire, il paraît évident que sa simulation sera plus lente que pour un gain.

4 Commande Prédictive Linéaire

4.1 Linéarisation

4.1.1. Rappelons le modèle linéarisé à un point d'équilibre :

$$\dot{x} = \frac{\partial f}{\partial x_{x_{ref}, u_{ref}}} (x - x_{ref}) + \frac{\partial f}{\partial u_{x_{ref}, u_{ref}}} (u - u_{ref})$$

4.1.2. En un point quelconque (x_t, u_t) , le terme $f(x_t, u_t)$ n'est a priori pas nul. Il faut donc ajouter :

$$\dot{x} = f(x_t, u_t) + \frac{\partial f}{\partial x_{x_t, u_t}} (x - x_t) + \frac{\partial f}{\partial u_{x_t, u_t}} (u - u_t)$$

4.1.3. Avec la commande LQR, nous linéarisons le modèle autour d'un point d'équilibre et nous calculons un correcteur statique pour ce modèle. Le MPC linéaire linéarise le modèle au moment où nous déterminons la trajectoire. Cela rend le comportement du système plus précis (cf partie 2.2, où nous avons vu qu'un LQR avec un état initial loin du point d'équilibre ne converge pas vers les valeurs de référence des états).

4.2 Mise en œuvre A

Nous modifions le code du contrôleur de la partie 3.2 :

```
% Extrait de Exp3_QTP_LMPC/MPC_controller.m
% fonction construct_problem

[...]
opti = casadi.Opti('conic');
opts = struct();
opts.print_time = false;
```

```
opts.printLevel = 'none';
opti.solver('qpoads',opts);
[...]
```

Nous définissons le modèle linéarisé comme une fonction CasADI :

```
% Extrait de Exp3_QTP_LMPC/parameters.m
```

```
f_lref = casadi.Function('ode',{x, u}, {A*(x-xref)+B*(u-uref)},...
{'state', 'input'}, {'dx'});
```

```
MPC.model = integrate_model(f_lref, MPC.dt, 'RK4');
```

La figure 12 donne les résultats de la simulation.

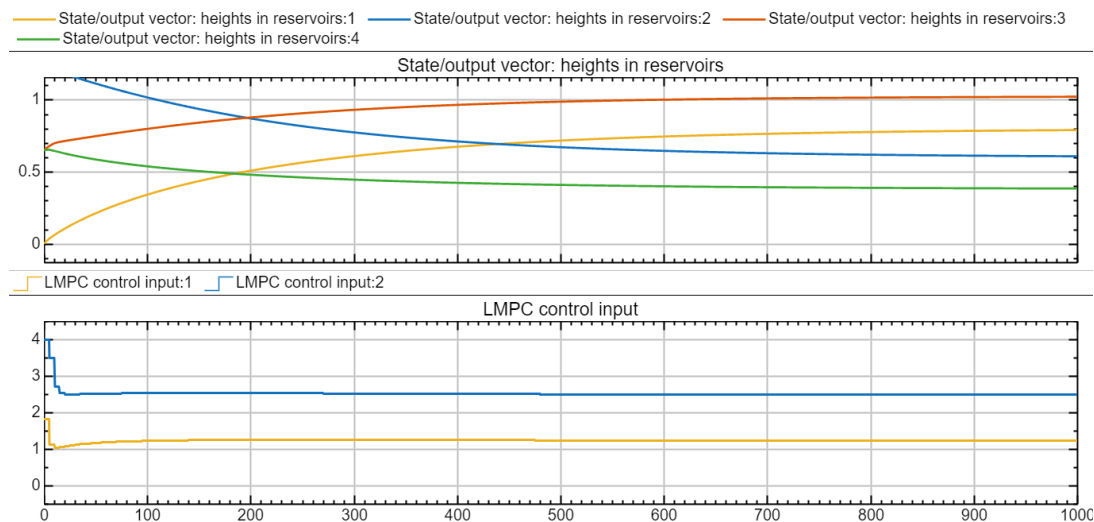


FIGURE 12 – Simulation LMPC méthode A

Les états convergent vers leur valeur de référence respective.

Modèle	Temps total d'exécution	Temps d'exécution du bloc régulateur
LMPC méthode A	34.8 s	0.939 s

TABLE 4 – Comparaison des temps d'exécution

4.3 Mise en œuvre B

Nous souhaitons réaliser un contrôleur qui calcule le modèle linéarisé à chaque itération. Nous modifions alors les codes précédents pour y inclure le point de fonctionnement de l'instant.

```
% Extrait de Exp4_QTP_LMPC/parameters.m
```

```
x0 = casadi.MX.sym('IC', nx, 1); % Define symbol for state
u0 = casadi.MX.sym('last_input', nu, 1); % Define symbol for input
PHI = f.jacobian(); % Define function return gradient of system
[A, B] = PHI(x0, u0, []); % Define problem: get Jacobien
% on operation point
```

```
% **Define model here**
f0 = f(x0, u0);
f_lact = casadi.Function('ode',{x, u, x0, u0}, {f0+A*(x-x0)+...
    B*(u-u0)},{'state', 'input', 'state_0', 'input_0'}, {'dx'});

MPC.model = integrate_model(f_lact, MPC.dt, 'RK4');
```

Extrait de Exp4_QTP_LMPC/MPC_controller.m

```
function [opti, x, x0, u0, xref, u] = construct_problem(obj)
    [...]
    % Define parameter variables
    u0 = opti.parameter(obj.nu, 1);
    [...]
    % Constraints
    x_next = obj.model(x(:, i), u(:, i), x0, u0);
    opti.subject_to(x(:, i+1) == x_next);
    [...]
```

```
function u = stepImpl(obj, xref, x)
    if isempty(obj.u)
        obj.u = zeros(obj.nu, 1);
    end
    u = obj.solve(x, xref);
end
```

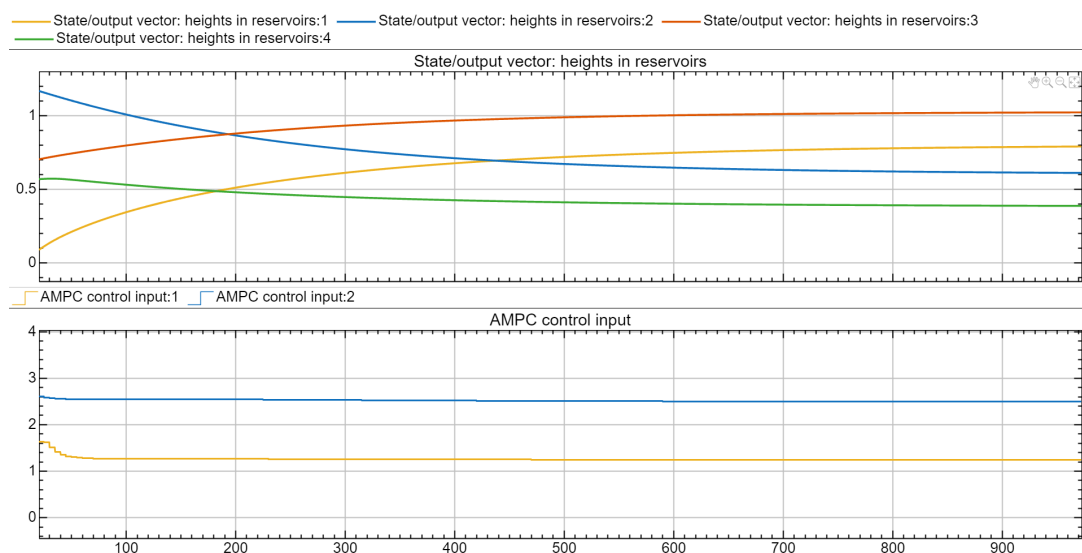


FIGURE 13 – Simulation LMPC méthode B

Modèle	Temps total d'exécution	Temps d'exécution du bloc régulateur
LMPC méthode B	32.49 s	0.926 s

TABLE 5 – Comparaison des temps d'exécution

4.2.1. D'après les figures 12 et 13 et les tableaux 4 et 5, les différences de temps de réponse et de temps d'exécution sont négligeables entre les deux contrôleurs. Cela s'explique par le fait que le système est "peu" non-linéaire.

4.2.2. En modifiant les hyperparamètres :

Modèle	hyperparamètre	t_{total}	$t_{régulateur}$	$t_{r,5\%}$
LMPC méthode B	$N = 10$	37.3 s	1.24 s	environ -40 s
LMPC méthode B	$dt = 8$	32.5 s	0.71 s	inchangé
NMPC	$N = 10$	37.3 s	1.24 s	inchangé
NMPC	$dt = 8$	46.5 s	6.98 s	environ -50 s
NMPC	$dt = 50$	20.31 s	0.67 s	environ -60s
LMPC méthode A	$dt = 8$	33.9 s	0.627 s	environ -30s
LMPC méthode A	$dt = 50$	27.6 s	0.26 s	environ -60s

TABLE 6 – Comparaison des temps d'exécution en fonction des hyperparamètres

D'après le tableau 6, il semble judicieux de sélectionner le contrôleur LMPC (méthode A) avec un pas de temps de 50 s. En effet, le modèle est "peu" non-linéaire comme expliqué plus tôt et le système est très lent, donc un pas de temps dt plus long permet de "voir plus loin" tout en conservant l'efficacité de l'horizon N fixe.

Cela est encore plus flagrant avec le NMPC, dont le temps de calcul diminue drastiquement pour un pas de temps de 50 s. Le temps de réponse est amélioré et le temps de calcul diminué.

4.2.3. Pour le système étudié, les différents contrôleurs proposent des performances plutôt similaires. En revanche, nous pouvons supposer que le AMPC sera adapté à une utilisation dans les processus lents, et le NMPC aux systèmes lents nécessitants de la précision.

Quelle que soit le système, la méthode prédictive permet de déterminer un signal de commande respectant des contraintes de saturation.